44. Design Web Analytics System

Key Terms:

Term	Full Name	What it does
HLL	HyperLogLog	Probabilistic unique count (~2% error, 12KB memory)
OLAP	Online Analytical Processing	Database optimized for aggregations (vs OLTP)
Columnar	Column-oriented storage	Stores data by column for faster analytics
sendBeacon	Browser API	Reliable data sending even on page unload
Lambda Architecture	Batch + Speed	Combines real-time stream with batch processing
TTL	Time To Live	Auto-delete data after retention period
Roll-up	Pre-aggregation	Summarize raw events into hourly/daily totals

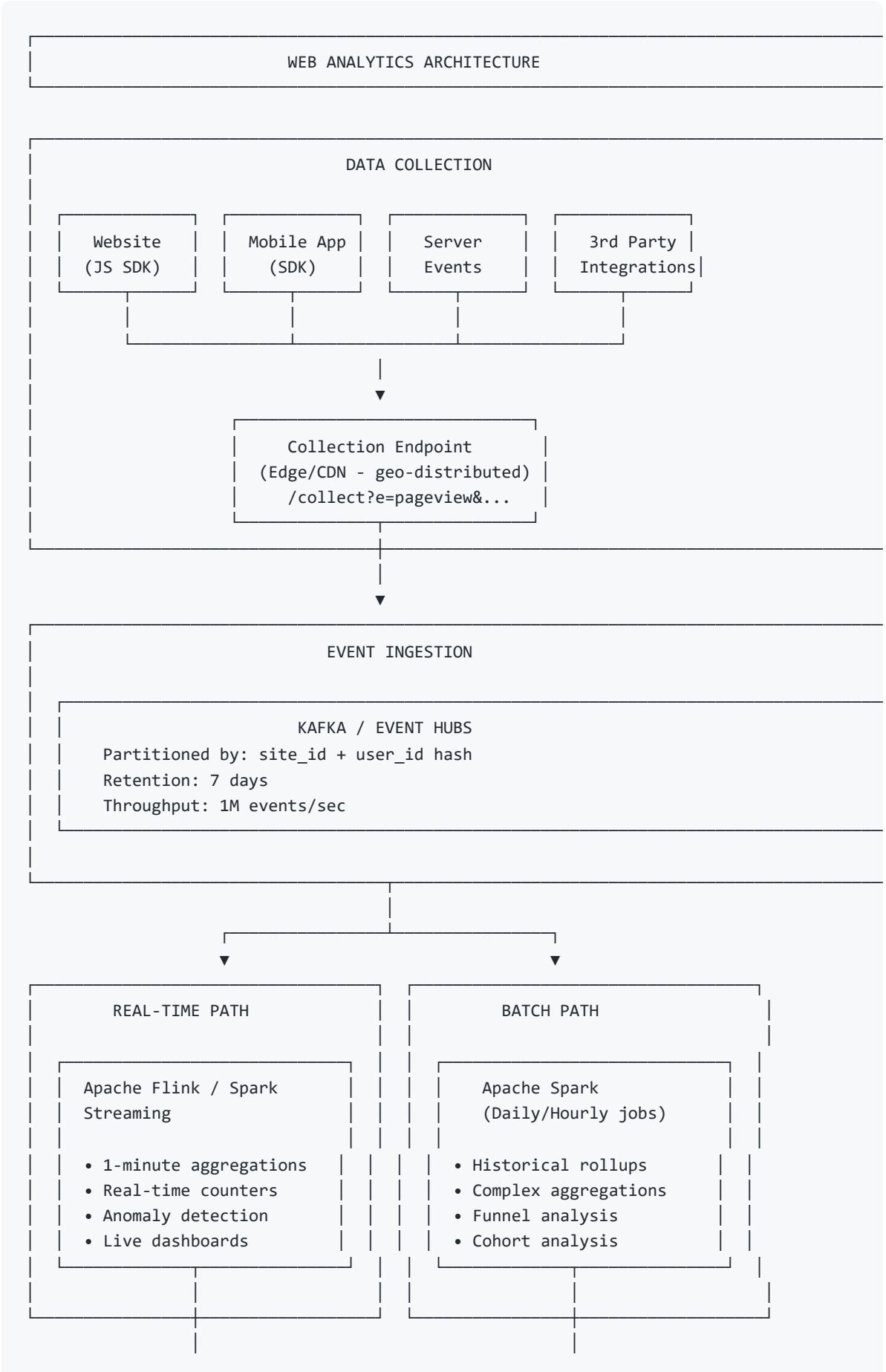
Problem Statement

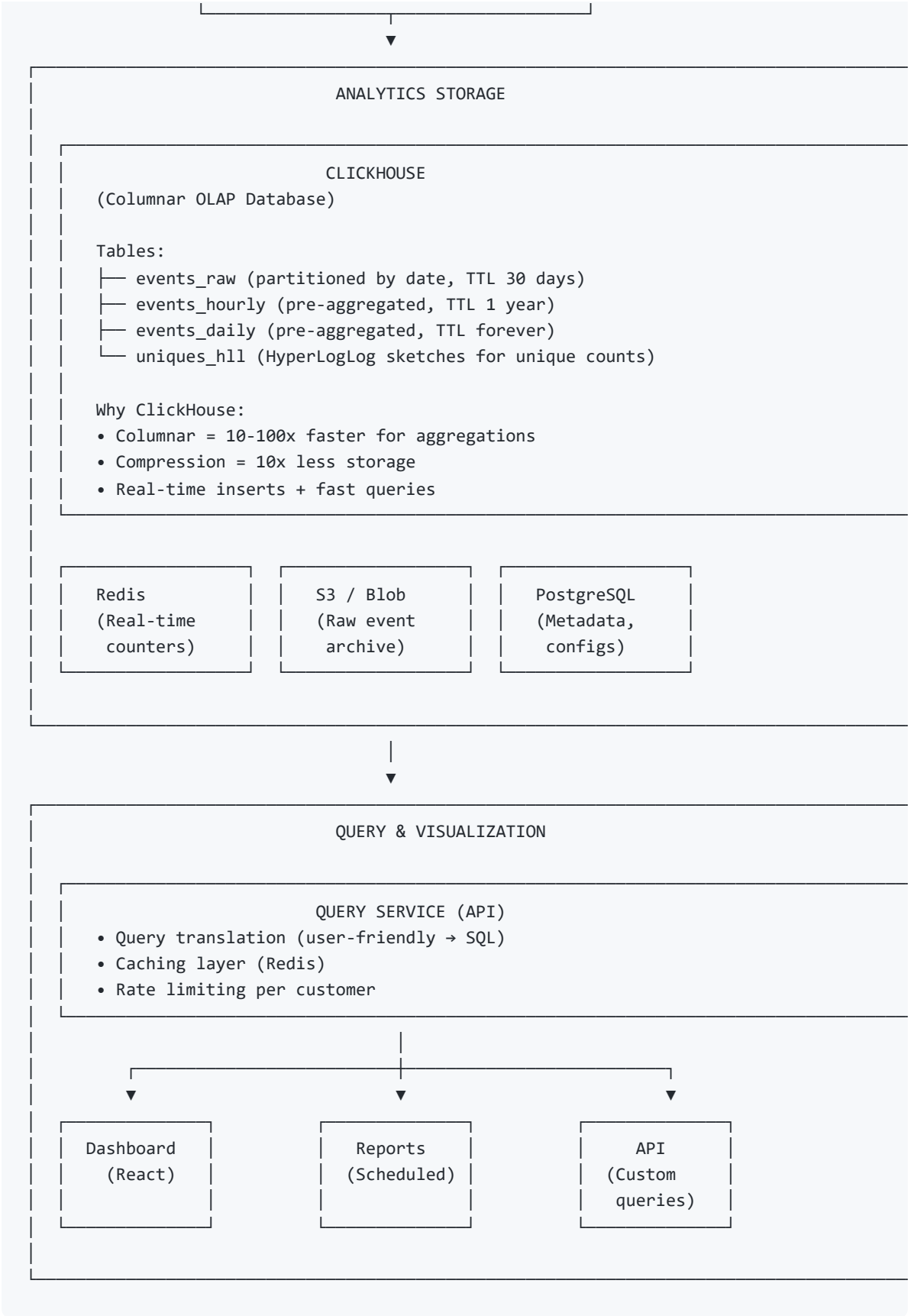
Design a real-time analytics system like Google Analytics that tracks user behavior (page views, clicks, conversions) and provides dashboards and reports.

Clarifying Questions to Ask

1. **Scale**: How many events per second? (100K-1M events/sec?)
2. **Latency**: Real-time dashboards or delayed OK? (seconds vs hours)
3. **Retention**: How long to keep raw data? (30 days? 1 year?)
4. **Queries**: What queries are most common? (page views, funnels, cohorts)
5. **Cardinality**: How many unique users/pages? (affects storage)
6. **Custom events**: Support custom event tracking?
7. **Attribution**: Need multi-touch attribution for marketing?

High-Level Design





API Design

Event Collection APIs

Method	Endpoint	Description
`POST`	`/api/v1/events`	Collect a single event
`POST`	`/api/v1/events/batch`	Collect batch of events
`GET`	`/api/v1/events/pixel.gif?...`	Tracking pixel (for email/no-JS)

Event Payload

```
POST /api/v1/events
{
  "site_id": "site_123",
  "event": "page_view",
  "url": "https://example.com/pricing",
  "referrer": "https://google.com",
  "user_id": "anon_456",
  "session_id": "sess_789",
  "timestamp": 1672531200000,
  "properties": {
    "viewport": "1920x1080",
    "browser": "Chrome 120"
  }
}
```

Query/Reporting APIs

Method	Endpoint	Description
`GET`	`/api/v1/analytics/pageviews?siteId={id}&from={date}&to={date}`	Get page view metrics
`GET`	`/api/v1/analytics/sessions?siteId={id}&groupBy=source`	Get session analytics
`GET`	`/api/v1/analytics/funnel?siteId={id}&steps={steps}`	Funnel analysis
`GET`	`/api/v1/analytics/realtime?siteId={id}`	Real-time active users
`POST`	`/api/v1/analytics/query`	Custom analytics query

Dashboard APIs

Method	Endpoint	Description

`POST`	`/api/v1/dashboards`	Create dashboard
`GET`	`/api/v1/dashboards/{dashId}`	Get dashboard with widgets

JavaScript SDK (Collection)

```
// analytics.js - Lightweight SDK (~2KB gzipped)
(function() {
  const ENDPOINT = 'https://collect.analytics.com/e';
  const queue = [];

  // Generate or retrieve anonymous ID
  const getAnonymousId = () => {
    let id = localStorage.getItem('_aid');
    if (!id) {
      id = 'a_' + Math.random().toString(36).substr(2, 16);
      localStorage.setItem('_aid', id);
    }
    return id;
  };

  // Core tracking function
  window.analytics = {
    track: function(event, properties = {}) {
      const payload = {
        e: event,                          // event name
        aid: getAnonymousId(),             // anonymous ID
        uid: this._userId,                 // user ID (if logged in)
        url: window.location.href,
        ref: document.referrer,
        ts: Date.now(),
        ua: navigator.userAgent,
        sr: `${screen.width}x${screen.height}`,
        ...properties
      };

      // Use sendBeacon for reliability (survives page unload)
      if (navigator.sendBeacon) {
        navigator.sendBeacon(ENDPOINT, JSON.stringify(payload));
      } else {
        // Fallback to image pixel
        new Image().src = `${ENDPOINT}?d=${encodeURIComponent(JSON.stringify(payload
      }
    },

    page: function(name, properties) {
      this.track('pageview', { page: name, ...properties });
    },

    identify: function(userId, traits) {
      this._userId = userId;
      this.track('identify', { traits });
```

```
    }
  };

  // Auto-track page views
  analytics.page();
})();
```

Event Schema

```
{
  "event_id": "evt_abc123",
  "site_id": "site_456",
  "event_type": "pageview",
  "anonymous_id": "a_xyz789",
  "user_id": "user_123",           // null if not logged in
  "session_id": "sess_def456",
  "timestamp": "2024-01-15T10:30:00Z",

  "page": {
    "url": "https://example.com/products/123",
    "path": "/products/123",
    "title": "Product Details",
    "referrer": "https://google.com"
  },

  "context": {
    "ip": "203.0.113.50",
    "user_agent": "Mozilla/5.0...",
    "locale": "en-US",
    "timezone": "America/Los_Angeles",
    "screen": "1920x1080",
    "viewport": "1200x800"
  },

  "properties": {
    "product_id": "123",
    "category": "electronics",
    "price": 299.99
  },

  "geo": {                         // Enriched from IP
    "country": "US",
    "region": "CA",
    "city": "San Francisco"
  }
}
```

ClickHouse Schema

```
-- Raw events table (hot data, 30-day TTL)
CREATE TABLE events_raw (
    event_id String,
    site_id String,
    event_type LowCardinality(String),
    anonymous_id String,
    user_id Nullable(String),
    session_id String,
    timestamp DateTime64(3),

    -- Page data
    url String,
    path String,
    referrer String,

    -- Context
    country LowCardinality(String),
    device_type LowCardinality(String),
    browser LowCardinality(String),
    os LowCardinality(String),

    -- Custom properties (flexible schema)
    properties String,  -- JSON string

    -- Partitioning
    date Date DEFAULT toDate(timestamp)
)
ENGINE = MergeTree()
PARTITION BY toYYYYMM(date)
ORDER BY (site_id, date, event_type, anonymous_id)
TTL date + INTERVAL 30 DAY;

-- Hourly aggregations (pre-computed for fast queries)
CREATE TABLE events_hourly (
    site_id String,
    hour DateTime,
    event_type LowCardinality(String),

    -- Counts
    event_count UInt64,
    unique_users AggregateFunction(uniq, String),
    unique_sessions AggregateFunction(uniq, String),

    -- Dimensions
    country LowCardinality(String),
    device_type LowCardinality(String),
    path String
)
ENGINE = AggregatingMergeTree()
PARTITION BY toYYYYMM(hour)
ORDER BY (site_id, hour, event_type, country, device_type, path);

-- Materialized view for automatic aggregation
CREATE MATERIALIZED VIEW events_hourly_mv TO events_hourly AS
```

```
    SELECT
        site_id,
        toStartOfHour(timestamp) AS hour,
        event_type,
        count() AS event_count,
        uniqState(anonymous_id) AS unique_users,
        uniqState(session_id) AS unique_sessions,
        country,
        device_type,
        path
    FROM events_raw
    GROUP BY site_id, hour, event_type, country, device_type, path;
```

Key Patterns

1. Pre-Aggregation (Roll-ups)

```
RAW EVENTS → MINUTE → HOUR → DAY → MONTH

┌─────────────────────────────────────────────────────────────┐
│  Query: "Page views last 7 days"                            │
│                                                             │
│  Without pre-agg: Scan 100M raw events (~10 seconds)        │
│  With pre-agg:    Scan 168 hourly rows (~10 milliseconds)   │
│                                                             │
│  1000x faster!                                              │
└─────────────────────────────────────────────────────────────┘
```

2. HyperLogLog for Unique Counts

```
┌─────────────────────────────────────────────────────────────┐
│  PROBLEM: Count unique visitors across 1 billion events     │
│                                                             │
│  Exact count: Store all 50M user IDs = 400MB memory         │
│  HyperLogLog: 12KB memory, ~2% error                        │
│                                                             │
│  ClickHouse: uniq() function uses HLL automatically         │
│                                                             │
│  SELECT uniq(anonymous_id) FROM events                      │
│  WHERE site_id = 'abc' AND date >= today() - 30             │
│                                                             │
│  Result: 1,234,567 (±2% = ±24,691)                          │
│  For dashboards, this accuracy is acceptable!               │
└─────────────────────────────────────────────────────────────┘
```

3. Session Stitching

```
┌─────────────────────────────────────────────────────────────┐
│  Events from same user need to be grouped into sessions     │
│                                                             │
│  SESSION RULES:                                             │
│    • Same anonymous_id                                      │
│    • Gap between events < 30 minutes                        │
│    • Same day (midnight = new session)                      │
│                                                             │
│  IMPLEMENTATION:                                            │
│    Flink: Session windows with 30-min gap                   │
│    Or: Compute session_id client-side                       │
└─────────────────────────────────────────────────────────────┘
```

Key Components

Component	Purpose	Technology
JS SDK	Client-side tracking	JavaScript (2KB)
Collection API	Receive events	Edge functions (Cloudflare/CDN)
Event Queue	Buffer and distribute	Kafka / Event Hubs
Stream Processor	Real-time aggregation	Apache Flink
Batch Processor	Historical analysis	Apache Spark
Analytics DB	Query storage	ClickHouse
Cache	Fast dashboard queries	Redis
Archive	Raw event backup	S3 / Blob Storage

Scalability Considerations

- **Collection at edge**: CDN endpoints reduce latency, handle spikes
- **Kafka partitioning**: Partition by site_id for parallelism
- **ClickHouse sharding**: Shard by site_id for multi-tenant isolation
- **Pre-aggregation**: 1000x query speedup for dashboards
- **TTL policies**: Raw data 30 days, aggregates forever
- **Sampling**: For very high volume, sample raw events (keep 10%)

Trade-offs

Decision	Option A	Option B	Recommendation

Storage	Row-based (PostgreSQL)	Columnar (ClickHouse)	**ClickHouse** - 10-100x faster
Unique counts	Exact	HyperLogLog (~2% error)	**HLL** - 1000x less memory
Processing	Real-time only	Lambda (real-time + batch)	**Lambda** - accuracy + speed
Collection	Server-side	Client-side (JS)	**Client-side** - richer context
Retention	Keep everything	TTL + sampling	**TTL** - cost effective

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	High-throughput event ingestion buffer partitioned by site_id	Tech Guide
ClickHouse	Columnar OLAP database for 10-100x faster aggregations	Tech Guide
HyperLogLog	Approximate unique visitor counts using 12KB instead of 400MB	Tech Guide
Flink/Spark	Real-time stream processing for minute aggregations and batch rollups	Tech Guide

How to Explain It Well

> **Opening**: "Web analytics needs to handle millions of events per second with real-time dashboards. I'll use a Lambda architecture with ClickHouse for fast OLAP queries."

> **Key talking points**:
>
> - "Lightweight JS SDK (~2KB) with sendBeacon for reliable collection"
> - "Kafka buffers events, Flink aggregates in real-time"
> - "ClickHouse columnar storage gives 10-100x faster aggregations than PostgreSQL"
> - "Pre-aggregate into hourly/daily rollups for O(1) dashboard queries"
> - "HyperLogLog for unique visitor counts - 12KB memory instead of 400MB, 2% error"
> - "TTL policies: raw events 30 days, aggregates forever"

Common Follow-up Questions

1. **How do you handle ad blockers?**
    - First-party domain for collection (analytics.yoursite.com)
    - Server-side tracking as backup
    - Accept some data loss (~10-20%)
2. **How do you ensure data accuracy?**
    - Deduplication by event_id
    - Late-arriving events handled by batch reprocessing
    - Checksums for data integrity
3. **How do you handle GDPR/privacy?**
    - Anonymize IPs (truncate last octet)
    - Cookie consent required for tracking
    - Data deletion API for right-to-be-forgotten
    - Configurable data retention per site